

UNIVERSITY OF ENGINEERING AND TECHNOLOGY
PESHAWAR, PAKISTAN

DATA STRUCTURES

Weeks 4

Lab Assignment

Name	Anas Khan
Department	Computer Science
Section	B
Registration Number	25PWCS1691
Submitted to	Sir Mudasir
University	University of Engineering and Technology, Peshawar, Pakistan

Exercises 1: Trace Binary Search by Hand

Exercises 1

Task 1:

Understand how the search interval changes.

Code:

A screenshot of a C++ IDE, likely Visual Studio Code, showing a file named 'Quest.cpp'. The code implements a binary search algorithm. It starts with a function signature 'int binarySearch(int arr[], int size, int target)' and includes necessary headers like <iostream> and <string>. The main function initializes an array 'arr' with values {4, 9, 15, 22, 31, 47, 56, 63, 71}, sets 'size' to 9, and calls 'binarySearch(arr, size, 47)'. The binary search function uses a while loop to find the target. It calculates 'mid' as (low + high) / 2 and checks if 'arr[mid]' equals the target. If not, it updates 'low' or 'high' based on whether the target is greater or less than the element at 'mid'. The function returns 'mid' if found, otherwise -1. The output window on the right shows the execution results, including the array elements and the search process for the target value 47.

Output:

```
42 |         cc mid << "1";  
-----  
Problem: Output Debug Console [Terminal] Ports  
# PS D:\DataStructures\Lab_4> cd "d:\DataStructures\Lab_4\" ; if ($?) { g++ Ques2.cpp -o Ques2 } ; if ($?) { .\Ques2 }  
  
Searching for: 42  
-----  
Low High Mid arr[mid] Decision  
0 8 4 31 Right  
5 8 6 58 Left  
5 5 5 47 Found  
Target 42 found at index 5.  
  
Searching for: 28  
-----  
Low High Mid arr[mid] Decision  
0 8 4 31 Left  
0 3 1 9 Right  
2 3 2 25 Right  
3 3 3 22 Left  
Target 28 not found. Search stops because low > high.  
PS D:\DataStructures\Lab_4 ]
```

Explanation:

Index: 0 1 2 3 4 5 6 7 8

Value: 4 9 15 22 31 47 58 63 71

1. Search for target = 47

Initially:

- low = 0
- high = 8
- mid = (low + high) / 2

Iteration low high mid arr[mid] Decision

1	0	8	4	31	47 > 31 → Right
2	5	8	6	58	47 < 58 → Left
3	5	5	5	47	Found

Result: Target 47 is found at **index 5**.

2. Search for target = 20

Initially:

- low = 0
- high = 8

Iteration low high mid arr[mid] Decision

1	0	8	4	31	20 < 31 → Left
2	0	3	1	9	20 > 9 → Right
3	2	3	2	15	20 > 15 → Right
4	3	3	3	22	20 < 22 → Left

After iteration 4:

- low = 3
- high = 2

Since **low > high**, the search interval is empty and the search **stops**.

Result: Target 20 is **not found** in the array.

Final observation

- For 47, binary search finds the target in **3 iterations** at index **5**.
- For 20, binary search performs **4 iterations**, then stops when low > high, confirming that 20 is not present.

Task 2:

Implement Iterative Binary Search & Convert the trace logic into working C++ code.

Code:

```

1 // Question 3
2 #include <iostream>
3 using namespace std;
4
5 // Iterative Binary Search
6 int binarySearch(int arr[], int n, int target, int &comparisons)
7 {
8     int low = 0;
9     int high = n - 1;
10    comparisons = 0;
11    while (low <= high)
12    {
13        int mid = (low + high) / 2;
14        // Count comparison with arr[mid]
15        comparisons++;
16        if (arr[mid] == target)
17        {
18            return mid;
19        }
20        else if (target < arr[mid])
21        {
22            high = mid - 1;
23        }
24        else
25        {
26            low = mid + 1;
27        }
28    }
29    return -1;
30 }
31
32 int main()
33 {
34     int n;
35     int arr[50];
36
37     // Read number of elements
38     cout << "Enter number of elements (maximum 50): ";
39     cin >> n;
40
41     // Validation
42     if (n <= 0 || n > 50)
43     {
44         cout << "Invalid number of elements." << endl;
45         return 0;
46     }
47
48     // Read sorted array
49     cout << "Enter " << n << " integers in ascending order." << endl;
50     for (int i = 0; i < n; i++)
51     {
52         cin >> arr[i];
53     }
54
55     // Read target
56     int target;
57     cout << "Enter target value: ";
58     cin >> target;
59
60     // Perform binary search
61     int comparisons;
62     int result = binarySearch(arr, n, target, comparisons);
63
64     // Display result
65     if (result != -1)
66     {
67         cout << "Target found at index: " << result << endl;
68     }
69     else
70     {
71         cout << "Not found" << endl;
72     }
73
74     cout << "Number of comparisons: " << comparisons << endl;
75     return 0;
76 }

```

Output:

```

28    {
29        low = mid + 1;
30    }
31 }
32
33 PS D:\Data Structures\Lab_4> cd "d:\Data Structures\Lab_4" ; if ($?) { g++ Quest3.cpp -o Quest3 } ; if ($?) { .\Quest3 }
Enter number of elements (maximum 50): 9
Enter 9 integers in ascending order:
4
5
9
22
31
47
58
63
71
Enter target value: 22
Target found at index: 2
Number of comparisons: 4
PS D:\Data Structures\Lab_4> cd "d:\Data Structures\Lab_4" ; if ($?) { g++ Quest3.cpp -o Quest3 } ; if ($?) { .\Quest3 }
Enter number of elements (maximum 50): 5
Enter 5 integers in ascending order:
21
34
56
78
98
Enter target value: 19
Not found
Number of comparisons: 2
PS D:\Data Structures\Lab_4>

```

Task 3:

Implement Recursive Binary Search and Understand the same decisions through recursive function calls. And Submit: C++ program + a short note comparing iterative loops with recursive calls.

Code:

```

1 // Recursive Binary Search
2 #include <iostream>
3 using namespace std;
4
5 // Recursive Binary Search
6 int recursiveBinarySearch(int arr[], int low, int high, int target)
7 {
8     // Base case: target not found
9     if (low > high)
10     {
11         return -1;
12     }
13
14     // Calculate middle index
15     int mid = (low + high) / 2;
16
17     // Print the mid index examined
18     cout << "Checking mid index: " << mid
19           << " (value = " << arr[mid] << ") " << endl;
20
21     // Target found
22     if (arr[mid] == target)
23     {
24         return mid;
25     }
26
27     // Search left half
28     if (target < arr[mid])
29     {
30         return recursiveBinarySearch(arr, low, mid - 1, target);
31     }
32
33     // Search right half
34     return recursiveBinarySearch(arr, mid + 1, high, target);
35 }
36
37 int main()
38 {
39     int arr[] = {4, 9, 31, 22, 33, 47, 54, 63, 71};
40     int n = 9;
41
42     // Test 1: Target is present
43     int target = 47;
44     cout << "Searching for: " << target << endl;
45     int result1 = recursiveBinarySearch(arr, 0, n - 1, target);
46     if (result1 != -1)
47     {
48         cout << "Target found at index: " << result1 << endl;
49     }
50     else
51     {
52         cout << "Not found" << endl;
53     }
54
55     // Test 2: Target is absent
56     int target = 20;
57     cout << "Searching for: " << target << endl;
58     int result2 = recursiveBinarySearch(arr, 0, n - 1, target);
59     if (result2 != -1)
60     {
61         cout << "Target found at index: " << result2 << endl;
62     }
63     else
64     {
65         cout << "Not found" << endl;
66     }
67
68     return 0;
69 }

```

Output:

```

16 // Print the mid index examined
17
18 PS D:\Data Structures\Lab_4> cd "d:\Data Structures\Lab_4" ; if ($?) { g++ Quest4.cpp -o Quest4 } ; if ($?) { .\Quest4 }
19
20 Searching for: 47
21 Checking mid index: 4 (value = 31)
22 Checking mid index: 6 (value = 58)
23 Checking mid index: 5 (value = 47)
24 Target found at index: 5
25
26 Searching for: 20
27 Checking mid index: 4 (value = 31)
28 Checking mid index: 1 (value = 9)
29 Checking mid index: 2 (value = 15)
30 Checking mid index: 3 (value = 22)
31 Not found
32
33 PS D:\Data Structures\Lab_4>

```

Explanation:**Iterative Binary Search:**

Iterative binary search uses a while loop to repeatedly update low, high, and mid. It does not create new function calls for each step, so it generally uses less memory.

Recursive Binary Search:

Recursive binary search calls the same function again with a smaller search range. Each call creates a new function-call stack frame. It can be easier to understand because the algorithm directly expresses the idea of searching one half of the array at a time.

Conclusion:

Both methods have the same time complexity of $O(\log n)$. Iterative binary search usually has lower memory overhead, while recursive binary search provides a cleaner representation of the divide-and-conquer logic.

Task 4:**Given Array:**

{7, 3, 5, 2, 8}

Bubble Sort is used to arrange the elements in ascending order. In each pass, adjacent elements are compared. If the left element is greater than the right element, they are swapped.

PASS 1**Initial Array:**

7 3 5 2 8

Comparison 1: Compare 7 and 3Since $7 > 3$, swap them.**After swap:**

3 7 5 2 8

Comparison 2: Compare 7 and 5Since $7 > 5$, swap them.**After swap:**

3 5 7 2 8

Comparison 3: Compare 7 and 2Since $7 > 2$, swap them.**After swap:**

3 5 2 7 8

Comparison 4: Compare 7 and 8Since $7 < 8$, no swap.**Array after Pass 1:**

3 5 2 7 [8]

Sorted suffix: [8]**PASS 2****Initial Array:**

3 5 2 7 [8]

Comparison 1: Compare 3 and 5Since $3 < 5$, no swap.**Array:**

3 5 2 7 8

Comparison 2: Compare 5 and 2Since $5 > 2$, swap them.

After swap:

3 2 5 7 8

Comparison 3: Compare 5 and 7

Since $5 < 7$, no swap.

Array after Pass 2:

3 2 5 [7] [8]

Sorted suffix: [7, 8]

PASS 3

Initial Array:

3 2 5 [7] [8]

Comparison 1: Compare 3 and 2

Since $3 > 2$, swap them.

After swap:

2 3 5 7 8

Comparison 2: Compare 3 and 5

Since $3 < 5$, no swap.

Array after Pass 3:

2 3 [5] [7] [8]

Sorted suffix: [5, 7, 8]

PASS 4

Initial Array:

2 3 [5] [7] [8]

Comparison 1: Compare 2 and 3

Since $2 < 3$, no swap.

Array after Pass 4:

[2] [3] [5] [7] [8]

Sorted suffix: [2, 3, 5, 7, 8]

The complete array is now sorted.

PASS-BY-PASS SUMMARY

Pass 1:

3 5 2 7 [8]

Sorted suffix: [8]

Pass 2:

3 2 5 [7] [8]

Sorted suffix: [7, 8]

Pass 3:

2 3 [5] [7] [8]

Sorted suffix: [5, 7, 8]

Pass 4:

[2] [3] [5] [7] [8]

Sorted suffix: [2, 3, 5, 7, 8]

FINAL SORTED ARRAY

{2, 3, 5, 7, 8}

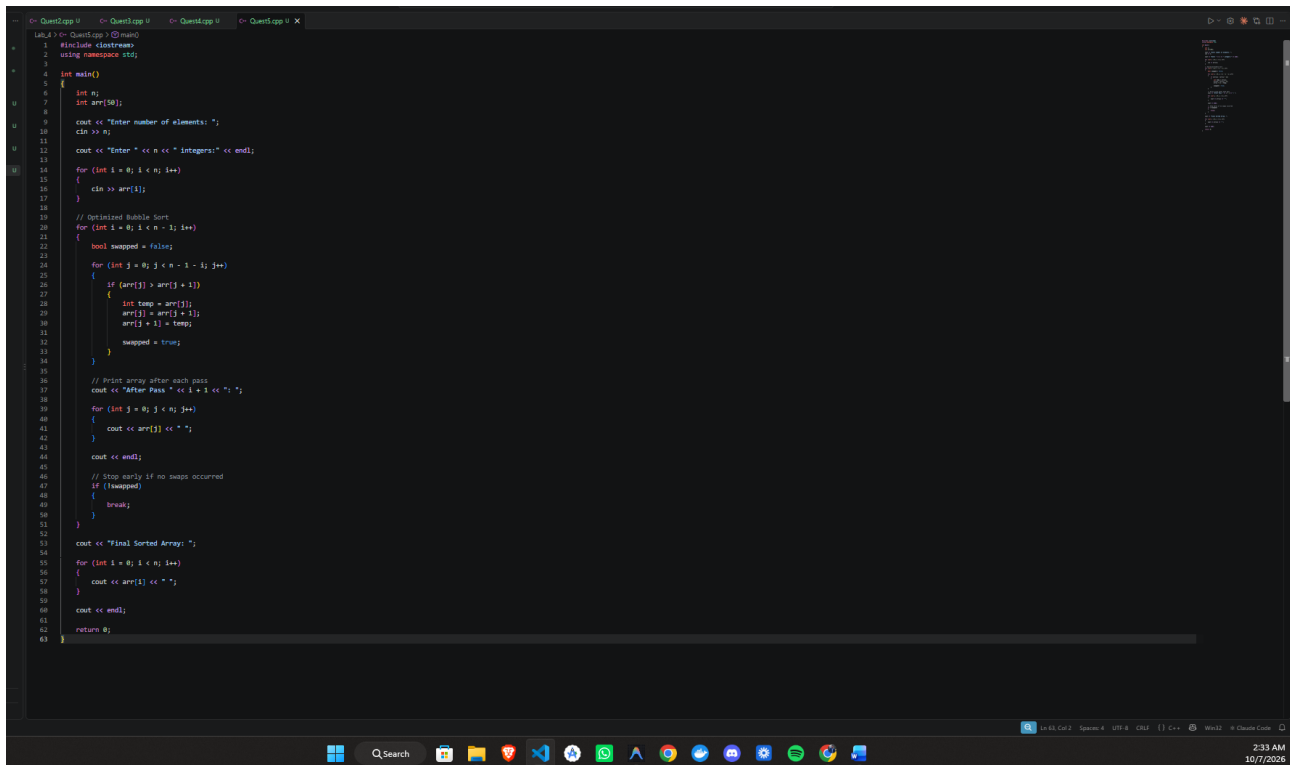
OBSERVATION

After every pass of Bubble Sort, the largest element from the unsorted portion moves to its correct position at the end of the array. Therefore, the sorted suffix increases by one element after each pass.

Task 5:

Implement Optimized Bubble Sort Build a correct bubble-sort program with early stopping. And Submit: C++ program + screenshot showing at least three passes.

Code:



```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      int arr[10];
8
9      cout << "Enter number of elements: ";
10     cin >> n;
11
12     cout << "Enter " << n << " integers:" << endl;
13
14     for (int i = 0; i < n; i++)
15     {
16         cin >> arr[i];
17     }
18
19     // Optimized Bubble Sort
20     for (int i = 0; i < n - 1; i++)
21     {
22         bool swapped = false;
23
24         for (int j = 0; j < n - 1 - i; j++)
25         {
26             if (arr[j] > arr[j + 1])
27             {
28                 int temp = arr[j];
29                 arr[j] = arr[j + 1];
30                 arr[j + 1] = temp;
31                 swapped = true;
32             }
33         }
34
35         // Print array after each pass
36         cout << "After Pass " << i + 1 << " : ";
37
38         for (int j = 0; j < n; j++)
39         {
40             cout << arr[j] << " ";
41         }
42
43         cout << endl;
44
45         // Stop early if no swap occurred
46         if (!swapped)
47         {
48             break;
49         }
50     }
51
52     cout << "Final Sorted Array: ";
53
54     for (int i = 0; i < n; i++)
55     {
56         cout << arr[i] << " ";
57     }
58
59     cout << endl;
60
61     return 0;
62 }

```

Output:



```

26     if (arr[j] > arr[j + 1])
27     {
28         int temp = arr[j];
29         arr[j] = arr[j + 1];
30         arr[j + 1] = temp;
31     }
32 }
33
34 // Print array after each pass
35 cout << "After Pass " << i + 1 << " : ";
36
37 for (int j = 0; j < n; j++)
38 {
39     cout << arr[j] << " ";
40 }
41
42 cout << endl;
43
44 // Stop early if no swap occurred
45 if (!swapped)
46 {
47     break;
48 }
49
50 }
51
52 cout << "Final Sorted Array: ";
53
54 for (int i = 0; i < n; i++)
55 {
56     cout << arr[i] << " ";
57 }
58
59 cout << endl;
60
61 return 0;
62 }

```

PS D:\Data Structures\Lab_4> cd "d:\Data Structures\Lab_4" ; if (\$?) { g++ Quest5.cpp -o Quest5 } ; if (\$?) { .\Quest5 }
 Enter number of elements: 4
 Enter 4 integers:
 1
 2
 3
 4
 After Pass 1: 1 2 3 4
 Final Sorted Array: 1 2 3 4
 PS D:\Data Structures\Lab_4> cd "d:\Data Structures\Lab_4" ; if (\$?) { g++ Quest5.cpp -o Quest5 } ; if (\$?) { .\Quest5 }
 Enter number of elements: 7
 Enter 7 integers:
 9
 7
 8
 6
 5
 4
 3
 After Pass 1: 7 8 6 5 4 3 9
 After Pass 2: 7 6 5 4 3 8 9
 After Pass 3: 6 5 4 3 7 8 9
 After Pass 4: 5 4 3 6 7 8 9
 After Pass 5: 4 3 5 6 7 8 9
 After Pass 6: 3 4 5 6 7 8 9
 Final Sorted Array: 3 4 5 6 7 8 9
 PS D:\Data Structures\Lab_4>

Task 6:

Measure Bubble Sort Work Objective: Connect the algorithm to comparisons and swaps. And Submit: A small results table plus the C++ program/output screenshots.

Code:

```

1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int arr[10];
7     int n;
8
9     cout << "Enter number of elements: ";
10    cin >> n;
11
12    cout << "Enter " << n << " integers: " << endl;
13
14    for (int i = 0; i < n; i++)
15    {
16        cin >> arr[i];
17    }
18
19    int comparisons = 0;
20    int swaps = 0;
21
22    // Optimized Bubble Sort
23    for (int i = 0; i < n - 1; i++)
24    {
25        bool swapped = false;
26
27        for (int j = 0; j < n - 1 - i; j++)
28        {
29            comparisons++;
30
31            if (arr[j] > arr[j + 1])
32            {
33                int temp = arr[j];
34                arr[j] = arr[j + 1];
35                arr[j + 1] = temp;
36
37                swapped = true;
38            }
39
40            if (!swapped)
41            {
42                break;
43            }
44        }
45
46        cout << "Sorted Array: ";
47
48        for (int i = 0; i < n; i++)
49        {
50            cout << arr[i] << " ";
51        }
52
53        cout << endl;
54        cout << "Total Comparisons: " << comparisons << endl;
55        cout << "Total Swaps: " << swaps << endl;
56
57        return 0;
58    }

```

Output:

```

11    cout << "Enter " << n << " integers: " << endl;
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Explanation

For the already sorted array, the first pass makes 5 comparisons and performs no swaps, so the optimized Bubble Sort stops immediately. For the reverse-sorted array, every comparison requires a swap, so the algorithm continues through all passes and performs more work

Task 7:

Practice tracking minIndex before coding. Submit: A complete selection-sort trace for all passes.

Given Array:

{29, 10, 14, 37, 13}

Selection Sort finds the **smallest value** in the unsorted portion and swaps it with the first element of that portion.

Initial Array

29 10 14 37 13

Pass 1

i = 0

Start:

29 10 14 37 13

^

i = 0

Find the minimum value from index 0 to 4:

- 29 → current minimum
- 10 → smaller than 29 → minIndex = 1
- 14 → greater than 10
- 37 → greater than 10
- 13 → greater than 10

Minimum value = 10

minIndex = 1

Swap arr[0] and arr[1]:

10 29 14 37 13

Sorted prefix:

[10] 29 14 37 13

Pass 2

i = 1

Start:

[10] 29 14 37 13

^

$i = 1$

Find the minimum from index 1 to 4:

- 29 → current minimum
- 14 → smaller → $\text{minIndex} = 2$
- 37 → greater than 14
- 13 → smaller → $\text{minIndex} = 4$

Minimum value = 13

$\text{minIndex} = 4$

Swap $\text{arr}[1]$ and $\text{arr}[4]$:

10 13 14 37 29

Sorted prefix:

[10] [13] 14 37 29

Pass 3

$i = 2$

Start:

[10] [13] 14 37 29

^

$i = 2$

Find the minimum from index 2 to 4:

- 14 → current minimum
- 37 → greater than 14
- 29 → greater than 14

Minimum value = 14

$\text{minIndex} = 2$

Since i and minIndex are the same, no actual swap is needed.

Array remains:

10 13 14 37 29

Sorted prefix:

[10] [13] [14] 37 29

Pass 4

$i = 3$

Start:

[10] [13] [14] 37 29

^

$i = 3$

Find the minimum from index 3 to 4:

- 37 → current minimum
- 29 → smaller → minIndex = 4

Minimum value = 29

minIndex = 4

Swap arr[3] and arr[4]:

10 13 14 29 37

Sorted prefix:

[10] [13] [14] [29] [37]

Complete Selection Sort Trace

<u>Pass</u>	<u>i</u>	<u>Minimum Value</u>	<u>minIndex</u>	<u>Swap</u>	<u>Array After Pass</u>
<u>1</u>	<u>0</u>	<u>10</u>	<u>1</u>	<u>29 ↔ 10</u>	<u>[10] 29 14 37 13</u>
<u>2</u>	<u>1</u>	<u>13</u>	<u>4</u>	<u>29 ↔ 13</u>	<u>[10] [13] 14 37 29</u>
<u>3</u>	<u>2</u>	<u>14</u>	<u>2</u>	<u>No swap</u>	<u>[10] [13] [14] 37 29</u>
<u>4</u>	<u>3</u>	<u>29</u>	<u>4</u>	<u>37 ↔ 29</u>	<u>[10] [13] [14] [29] [37]</u>

Final Sorted Array

{10, 13, 14, 29, 37}

Observation: After every pass, the smallest element from the unsorted portion is placed at the correct position. Therefore, the sorted prefix grows by one element after each pass.

Task 8:

Implement minimum selection and placement. And submit C++ program + output showing the selected minimum on every pass.

Code:

```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      int arr[50];
8
9      cout << "Enter number of elements: ";
10     cin >> n;
11
12     cout << "Enter " << n << " integers: " << endl;
13
14     for (int i = 0; i < n; i++)
15     {
16         cin >> arr[i];
17     }
18
19     // Selection Sort
20     for (int i = 0; i < n - 1; i++)
21     {
22         int minIndex = i;
23
24         // Find minimum element
25         for (int j = i + 1; j < n; j++)
26         {
27             if (arr[j] < arr[minIndex])
28             {
29                 minIndex = j;
30             }
31         }
32
33         // Print selected minimum before swap
34         cout << "Pass " << i + 1 << ": ";
35         cout << "Minimum = " << arr[minIndex];
36         cout << ", Index = " << minIndex << endl;
37
38         // Swap minimum with current element
39         if (minIndex != i)
40         {
41             int temp = arr[i];
42             arr[i] = arr[minIndex];
43             arr[minIndex] = temp;
44         }
45
46         // Print array after each pass
47         cout << "Array after Pass " << i + 1 << ": ";
48         for (int j = 0; j < n; j++)
49         {
50             cout << arr[j] << " ";
51         }
52         cout << endl;
53     }
54
55     // Final sorted array
56     cout << "Final Sorted Array: ";
57     for (int i = 0; i < n; i++)
58     {
59         cout << arr[i] << " ";
60     }
61     cout << endl;
62     return 0;
63 }

```

Output:

```

PS D:\Data Structures\Lab_4> cd "D:\Data Structures\Lab_4" ; if ($?) { g++ Quest7.cpp -o Quest7 } ; if ($?) { .\Quest7 }
Enter number of elements: 6
Enter 6 integers:
11
22
33
44
55
66
Pass 1: Minimum = 11, Index = 0
Array after Pass 1: 11 22 33 44 55 66
Pass 2: Minimum = 22, Index = 1
Array after Pass 2: 11 22 33 44 55 66
Pass 3: Minimum = 33, Index = 2
Array after Pass 3: 11 22 33 44 55 66
Pass 4: Minimum = 44, Index = 3
Array after Pass 4: 11 22 33 44 55 66
Pass 5: Minimum = 55, Index = 4
Array after Pass 5: 11 22 33 44 55 66
Final Sorted Array: 11 22 33 44 55 66
PS D:\Data Structures\Lab_4>

```

Task 9:

Selection Sort in Descending Order, Objective: Adapt the same structure by changing the selection rule. Submit: Modified program + output + comparison/swap counts.

Code:

```

1 // Lab_4.cpp
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int arr[] = {11, 7, 29, 3, 15, 8};
8     int n = 6;
9
10    int comparisons = 0;
11    int swaps = 0;
12
13    // Selection Sort in Descending order
14    for (int i = 0; i < n - 1; i++)
15    {
16        int maxIndex = i;
17
18        // Find maxIndex element
19        for (int j = i + 1; j < n; j++)
20        {
21            comparisons++;
22            if (arr[j] > arr[maxIndex])
23            {
24                maxIndex = j;
25            }
26        }
27
28        // Swap maxIndex with current element
29        if (maxIndex != i)
30        {
31            int temp = arr[i];
32            arr[i] = arr[maxIndex];
33            arr[maxIndex] = temp;
34            swaps++;
35        }
36
37        // Print array after each pass
38        cout << "After Pass " << i + 1 << ": ";
39        for (int j = 0; j < n; j++)
40        {
41            cout << arr[j] << " ";
42        }
43        cout << endl;
44    }
45
46    // Print final array
47    cout << "Final Array in Descending Order: ";
48    for (int i = 0; i < n; i++)
49    {
50        cout << arr[i] << " ";
51    }
52    cout << endl;
53
54    cout << "Total Comparisons: " << comparisons << endl;
55    cout << "Total Swaps: " << swaps << endl;
56
57    return 0;
58 }

```

Output:

```

1 // Lab_4.cpp
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int arr[] = {11, 7, 29, 3, 15, 8};
8     int n = 6;
9
10    int comparisons = 0;
11    int swaps = 0;
12
13    // Selection Sort in Descending order
14    for (int i = 0; i < n - 1; i++)
15    {
16        int maxIndex = i;
17
18        // Find maxIndex element
19        for (int j = i + 1; j < n; j++)
20        {
21            comparisons++;
22            if (arr[j] > arr[maxIndex])
23            {
24                maxIndex = j;
25            }
26        }
27
28        // Swap maxIndex with current element
29        if (maxIndex != i)
30        {
31            int temp = arr[i];
32            arr[i] = arr[maxIndex];
33            arr[maxIndex] = temp;
34            swaps++;
35        }
36
37        // Print array after each pass
38        cout << "After Pass " << i + 1 << ": ";
39        for (int j = 0; j < n; j++)
40        {
41            cout << arr[j] << " ";
42        }
43        cout << endl;
44    }
45
46    // Print final array
47    cout << "Final Array in Descending Order: ";
48    for (int i = 0; i < n; i++)
49    {
50        cout << arr[i] << " ";
51    }
52    cout << endl;
53
54    cout << "Total Comparisons: " << comparisons << endl;
55    cout << "Total Swaps: " << swaps << endl;
56
57    return 0;
58 }

```

Task 10:

Visualize Keys and Shifts: Understand the movement that happens before insertion.

Given Array:

{8, 4, 6, 2, 7}

Insertion Sort takes one value at a time (called the key), shifts larger values one position to the right, and then places the key in its correct position.

INITIAL ARRAY

8 4 6 2 7

Initially, the sorted prefix contains only:

[8]

PASS 1

Index: 1

Key: 4

Compare 4 with 8.

Since $8 > 4$, shift 8 one position to the right.

Shift:

$8 \rightarrow \text{right}$

Array after shift:

8 8 6 2 7

Place the key 4 in its correct position.

Array after placing key:

4 8 6 2 7

Sorted prefix:

[4, 8]

PASS 2

Index: 2

Key: 6

Compare 6 with 8.

Since $8 > 6$, shift 8 one position to the right.

Shift:

$8 \rightarrow \text{right}$

Array after shift:

4 8 8 2 7

Compare 6 with 4.

Since $4 < 6$, stop shifting.

Place the key 6.

Array after placing key:

4 6 8 2 7

Sorted prefix:

[4, 6, 8]

PASS 3

Index: 3

Key: 2

Compare 2 with 8.

Since $8 > 2$, shift 8 one position to the right.

Array after shift:

4 6 8 8 7

Compare 2 with 6.

Since $6 > 2$, shift 6 one position to the right.

Array after shift:

4 6 6 8 7

Compare 2 with 4.

Since $4 > 2$, shift 4 one position to the right.

Array after shift:

4 4 6 8 7

There are no more elements to the left.

Place the key 2 at index 0.

Array after placing key:

2 4 6 8 7

Sorted prefix:

[2, 4, 6, 8]

PASS 4

Index: 4

Key: 7

Compare 7 with 8.

Since $8 > 7$, shift 8 one position to the right.

Array after shift:

2 4 6 8 8

Compare 7 with 6.

Since $6 < 7$, stop shifting.

Place the key 7.

Array after placing key:

2 4 6 7 8

Sorted prefix:
[2, 4, 6, 7, 8]

COMPLETE DRY RUN SUMMARY

Pass 1:

Key = 4

Shifted = 8

Array = 4 8 6 2 7

Sorted prefix = [4, 8]

Pass 2:

Key = 6

Shifted = 8

Array = 4 6 8 2 7

Sorted prefix = [4, 6, 8]

Pass 3:

Key = 2

Shifted = 8, 6, 4

Array = 2 4 6 8 7

Sorted prefix = [2, 4, 6, 8]

Pass 4:

Key = 7

Shifted = 8

Array = 2 4 6 7 8

Sorted prefix = [2, 4, 6, 7, 8]

FINAL SORTED ARRAY

{2, 4, 6, 7, 8}

OBSERVATION

In each insertion, the key is temporarily stored while larger elements in the sorted prefix are shifted one position to the right. After all required shifts are completed, the key is placed into the empty position, increasing the size of the sorted prefix by one element.

Task 11:

Translate shifting and insertion into C++. C++ program + output showing every insertion step and the shift count.

Code:

```

1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7     int arr[10];
8     int shifts = 0;
9
10    cout << "Enter number of elements: ";
11    cin >> n;
12
13    cout << "Enter " << n << " integers:" << endl;
14
15    for (int i = 0; i < n; i++)
16    {
17        cin >> arr[i];
18    }
19
20    // Insertion Sort
21    for (int i = 1; i < n; i++)
22    {
23        int key = arr[i];
24        int j = i - 1;
25
26        // Shift elements greater than key
27        while (j >= 0 && arr[j] > key)
28        {
29            arr[j + 1] = arr[j];
30            j--;
31            shifts++;
32        }
33
34        // Place key in correct position
35        arr[j + 1] = key;
36
37        // Print array after each insertion
38        cout << "After inserting key " << key << ": ";
39
40        for (int k = 0; k < n; k++)
41        {
42            cout << arr[k] << " ";
43        }
44
45        cout << endl;
46    }
47
48    // Print final sorted array
49    cout << "Final Sorted Array: ";
50
51    for (int i = 0; i < n; i++)
52    {
53        cout << arr[i] << " ";
54    }
55
56    cout << endl;
57
58    // Print total shifts
59    cout << "Total Shifts: " << shifts << endl;
60
61    return 0;
62 }

```

Terminal Output:

```

PS D:\DataStructures\Lab_4> cd "d:\DataStructures\Lab_4" ; if ($?) { g++ Quest8.cpp -o Quest8 } ; if ($?) { .\Quest8 }
Total Shifts: 8
PS D:\DataStructures\Lab_4>

```

Output:

```

35 // Place key in correct position
36 arr[j + 1] = key;
37
38 // Print array after each insertion
39 cout << "After inserting key " << key << ": ";
40
41 for (int k = 0; k < n; k++)

```

Terminal Output:

```

PS D:\DataStructures\Lab_4> cd "d:\DataStructures\Lab_4" ; if ($?) { g++ Quest9.cpp -o Quest9 } ; if ($?) { .\Quest9 }
Enter number of elements: 5
Enter 5 integers:
13
66
77
88
99
After inserting key 66: 13 66 77 88 99
After inserting key 77: 13 66 77 88 99
After inserting key 88: 13 66 77 88 99
After inserting key 99: 13 66 77 88 99
Final Sorted Array: 13 66 77 88 99
Total Shifts: 8
PS D:\DataStructures\Lab_4>

```

Task 12:

Combine insertion sort and binary search into one complete workflow. One complete C++ program + screenshots of both test cases.

Code:

```

1 // Lab 4 > C++ > Quest10.cpp > @ main
2 #include <iostream>
3 using namespace std;
4
5 // Insertion Sort
6 void insertionSort(int arr[], int n)
7 {
8     for (int i = 1; i < n; i++)
9     {
10         int key = arr[i];
11         int j = i - 1;
12         while (j >= 0 && arr[j] > key)
13         {
14             arr[j + 1] = arr[j];
15             j--;
16         }
17         arr[j + 1] = key;
18     }
19 }
20
21 // Iterative Binary Search
22 int binarySearch(int arr[], int n, int target)
23 {
24     int low = 0;
25     int high = n - 1;
26     while (low <= high)
27     {
28         int mid = (low + high) / 2;
29         if (arr[mid] == target)
30         {
31             return mid;
32         }
33         else if (target < arr[mid])
34         {
35             high = mid - 1;
36         }
37         else
38         {
39             low = mid + 1;
40         }
41     }
42     return -1;
43 }
44
45 int main()
46 {
47     int n;
48     int arr[50];
49     cout << "Enter number of elements: ";
50     cin >> n;
51     cout << "Enter " << n << " unsorted integers: " << endl;
52     for (int i = 0; i < n; i++)
53     {
54         cin >> arr[i];
55     }
56     // Print original array
57     cout << "Original Array: ";
58     for (int i = 0; i < n; i++)
59     {
60         cout << arr[i] << " ";
61     }
62     cout << endl;
63     // Sort using Insertion Sort
64     insertionSort(arr, n);
65     // Print sorted array
66     cout << "Sorted Array: ";
67     for (int i = 0; i < n; i++)
68     {
69         cout << arr[i] << " ";
70     }
71     cout << endl;
72     // Read target
73     int target;
74     cout << "Enter target to search: ";
75     cin >> target;
76     // Binary Search
77     int result = binarySearch(arr, n, target);
78     if (result != -1)
79     {
80         cout << "Target found at index: " << result << endl;
81     }
82     else
83     {
84         cout << "Not found" << endl;
85     }
86     return 0;
87 }

```

```

88 // Lab 4 > C++ > Quest10.cpp > @ main
89 // Print original array
90 cout << "Original Array: ";
91 for (int i = 0; i < n; i++)
92 {
93     cout << arr[i] << " ";
94 }
95 cout << endl;
96 // Sort using Insertion Sort
97 insertionSort(arr, n);
98 // Print sorted array
99 cout << "Sorted Array: ";
100 for (int i = 0; i < n; i++)
101 {
102     cout << arr[i] << " ";
103 }
104 cout << endl;
105 // Read target
106 int target;
107 cout << "Enter target to search: ";
108 cin >> target;
109 // Binary Search
110 int result = binarySearch(arr, n, target);
111 if (result != -1)
112 {
113     cout << "Target found at index: " << result << endl;
114 }
115 else
116 {
117     cout << "Not found" << endl;
118 }
119 return 0;

```

Output:

The screenshot displays a C++ program in Visual Studio Code, implementing a linked list with insertion and search operations. The program is divided into three main sections, each starting with a prompt like "PS D:\Data\Structures\Lab_4> cd \"d:\Data\Structures\Lab_4\" ; if (\$?) { g++ Quest9.cpp -o Quest9 } ; if (\$?) { .\Quest9 }".

Section 1: Insertion and Sorting

```
# PS D:\Data\Structures\Lab_4> cd "d:\Data\Structures\Lab_4\" ; if ($?) { g++ Quest9.cpp -o Quest9 } ; if ($?) { .\Quest9 }
Enter number of elements: 5
Enter 5 integers:
33
66
77
88
99
After inserting key 66: 33 66 77 88 99
After inserting key 77: 33 66 77 88 99
After inserting key 88: 33 66 77 88 99
After inserting key 99: 33 66 77 88 99
Final Sorted Array: 33 66 77 88 99
Total Nodes: 8
```

Section 2: Search and Deletion

```
# PS D:\Data\Structures\Lab_4> cd "d:\Data\Structures\Lab_4\" ; if ($?) { g++ Quest10.cpp -o Quest10 } ; if ($?) { .\Quest10 }
Enter number of elements: 6
Enter 6 unsorted integers:
2
5
9
8
1
2
Original Array: 2 5 9 8 1 2
Sorted Array: 0 1 2 2 5 9
Enter target to search: 5
Target found at index: 4
# PS D:\Data\Structures\Lab_4> cd "d:\Data\Structures\Lab_4\" ; if ($?) { g++ Quest10.cpp -o Quest10 } ; if ($?) { .\Quest10 }
Enter number of elements: 4
Enter 4 unsorted integers:
1
4
5
8
Original Array: 1 4 5 8
Sorted Array: 1 4 5 8
Enter target to search: 10
Not found
# PS D:\Data\Structures\Lab_4>
```

The bottom of the image shows the Windows taskbar with various application icons and the system clock indicating 3:27 AM on 10/7/2026.